

PA1 : Benchmark Translator

Student : 卓祐晟

Student ID : 111521051

Date : 2024/03/17

Course : CAD for VLSI Design

Supervisor : Professor 陳聿廣

Outline

- PA1 : Benchmark Translator
 - Outline
 - 1. Assignment Requirement
 - 2. How to Compile and Execute
 - 3. Pseudocode
 - 4. My Data Structure and the Program Details
 - Program Idea
 - Read iscfile
 - How to deal with the type "from"
 - Output Verilog Code
 - 5. The Hardness I Encountered
 - How to store the data of From type?
 - 6. My Suggestion
 - 7. Reference

1. Assignment Requirement

You are requested to implement a C++ program that can take the ISCAS'85 netlist descriptions in its original format and translate them to the corresponding gate-level Verilog format.

2. How to Compile and Execute

Step1. Select the folder and use gedit to create 111521051_PA1.cpp

```
Last login: Wed Mar 13 19:18:51 2024 from 140.115.70.44
[s111521051@cad ~]$ ls
PA1
[s111521051@cad ~]$ cd PA1
[s111521051@cad ~/PA1]$ gedit 111521051_PA1.cpp
```

Step2. The same way goes for c17.isc, c17_testbench.v

/home/CAD112/s111521051/PA1/				
Name	Size (KB)	Last modified	Owner	Group
..				
111521051_PA1	58	2024-03-20...	s111521051	s111521051
111521051_PA1.cpp	9	2024-03-20...	s111521051	s111521051
c17.isc	1	2024-03-20...	s111521051	s111521051
c17_testbench.v	4	2024-03-20...	s111521051	s111521051
c432.isc	18	2024-03-13...	s111521051	s111521051
c432.v	10	2024-03-20...	s111521051	s111521051
c432_testbench.v	8	2024-03-13...	s111521051	s111521051
ncverilog.history	1	2024-03-20...	s111521051	s111521051
ncverilog.log	3	2024-03-20...	s111521051	s111521051
novas_dump.log	4	2024-03-20...	s111521051	s111521051

Step3. Compile with C++11 standards

```
[s111521051@cad ~/PA1]$ g++ -std=c++11 111521051_PA1.cpp -o 111521051_PA1
```

Step4. Execute the program by the command like the following

```
[s111521051@cad ~/PA1]$ ./111521051_PA1 c17.isc c17.v
```

If you enter a incorrect command type, the following reminder will appear:

```
[s111521051@cad ~/PA1]$ ./111521051_PA1
Input Type : ./111521051_PA1iscfile verilogfile
```

Step5. Enter the following command

```
[s111521051@cad ~/PA1]$ source /usr/cad/cadence/CIC/incisiv.cshrc
[s111521051@cad ~/PA1]$ source /usr/cad/synopsys/CIC/verdi.cshrc
[s111521051@cad ~/PA1]$ ncverilog +access+r c17_testbench.v c17.v
```

Test result of module c17

```

ncverilog: 15.20-s039: (c) Copyright 1995-2017 Cadence Design Systems, Inc.
file: c17_testbench.v
    module worklib.c17_tb:v
        errors: 0, warnings: 0
file: c17.v
    module worklib.c17:v
        errors: 0, warnings: 0
        Caching library 'worklib' ..... Done
    Elaborating the design hierarchy:
    Building instance overlay tables: ..... Done
    Generating native compiled code:
        worklib.c17_tb:v <0x4daf68c6>
            streams: 11, words: 30377
    Building instance specific data structures.
    Loading native compiled code: ..... Done
    Design hierarchy summary:
            Instances  Unique
    Modules:                2      2
    Primitives:              6      1
    Registers:               6      6
    Scalar wires:            5      -
    Initial blocks:          1      1
    Pseudo assignments:      5      5
    Simulation timescale: 1ps
    Writing initial simulation snapshot: worklib.c17_tb:v
    Loading snapshot worklib.c17_tb:v ..... Done
*Verdi* Loading libsscore_ius152.so
ncsim> source /usr/cad/cadence/INCISIV/cur/tools/inca/files/ncsimrc
ncsim> run
    input pattern = 00000--> golden value = 00
    your answer = 00
    input pattern = 10101--> golden value = 11
    your answer = 11
    input pattern = 01010--> golden value = 11
    your answer = 11
    input pattern = 11011--> golden value = 11
    your answer = 11
    input pattern = 11111--> golden value = 10
    your answer = 10
    CORRECT!!!

      ...XXXXXXXXXX..
    .XXXXXX.....XXXXX.
      XXXXX.....XXXXX.
    .XXX.....XX.
      XX.  ..    ..XX
    .XX.. .XXX. .XXX. .XX
      XXX.....XXXXX. XXXXX ..XX.
    ..XXX.....XXX.....XXX.....XXX
  XXXXXXXX..XXXXXXXXXXXXXXXXXXXXX.....XXXX.
  XXX.  XXXX..XXXXX.....XXXXX.. ..XXXXXXXXXX.
  XX.  XXX.....XXXXXXXXXXXXX.....XXXXXX .XXX.
  .XXX. .XX.. ..XXXXXXXXXXXXX.....XXX  XX.
    .XXXXX. .XX.  ..XXX  .XXX.
      ...XXXXXXX.  ...XXX ..XXXXXXX.
        ..XXXXX.  ....XXXXXXXXXX...
          .XXXX.....XXXXX.
            XXXXXXXXXXXXXXXXXXXX
              XXXXXXXXXXXXXXXXXXXX
                XXXXXXXXXXXXXXXXXXXX

```


Test result of module c432

```
[s111521051@cad ~/PA1]$ ncverilog +access+r c432_testbench.v c432.v
ncverilog: 15.20-s039: (c) Copyright 1995-2017 Cadence Design Systems, Inc.
file: c432_testbench.v
    module worklib.c432_tb:v
        errors: 0, warnings: 0
file: c432.v
    module worklib.c432:v
        errors: 0, warnings: 0
        Caching library 'worklib' ..... Done
    Elaborating the design hierarchy:
    Building instance overlay tables: ..... Done
    Generating native compiled code:
        worklib.c432_tb:v <0x56a7da7e>
        streams: 42, words: 75797
    Building instance specific data structures.
    Loading native compiled code: ..... Done
    Design hierarchy summary:
        Instances  Unique
    Modules:         2      2
    Primitives:      160     5
    Registers:       37     37
    Scalar wires:    36      -
    Initial blocks:   1      1
    Pseudo assignments: 36    36
    Simulation timescale: 1ps
    Writing initial simulation snapshot: worklib.c432_tb:v
Loading snapshot worklib.c432_tb:v ..... Done
*Verdi* Loading libsscore_ius152.so
ncsim> source /usr/cad/cadence/INCISIV/cur/tools/inca/files/ncsimrc
ncsim> run
input pattern = 111110000011111000001111100000111110--> golden value = 1001011
your answer = 1001011
input pattern = 11001111111111111110000000000001100 --> golden value = 1111000
your answer = 1111000
input pattern = 000001111111111111100000000000000000 --> golden value = 1001111
your answer = 1001111
input pattern = 0000011111111111111110000000000001111 --> golden value = 1001111
your answer = 1001111
input pattern = 1111111111111111110011000000000000000 --> golden value = 1111011
your answer = 1111011
CORRECT!!!

      .XXXXXXXXXXXXX..
    .XXXXXXXXX.....XXXXXXXXX.
      XXXXX....      .....XXXX.
    .XXX.....      .....XX.
      .XX.  ..      ...  .XX
    .XX..  .XXX.  .XXX.  .XX
    XXX....XXXXX.  XXXXX  ..XX.
  ..XXX.....XXX.....XXX.....XXX
XXXXXXXXXX..XXXXXXXXXXXXXXXXXXXXXXXXX.....XXXX.
XXX.  XXXX..XXXXX.....XXXXXXXXX.. ..XXXXXXXXXX.
XX.  XXX.....XXXXXXXXXXXXX.....XXXXXX  .XXX.
.XXX. .XX.  ..      .....XXX  XX.
  .XXXXX. .XX.  ..      .....XXX  .XXX.
    .XXXXXXXXX.      ...XXX  .XXXXXXXXX.
      .XXXXX.      ....XXXXXXXXXX...
    .XXXX.....XXXXX.
      .XXXXXXXXXXXXXXXXXXXXX.
      XXXXXXXXXXXXXXXXXXXXXXXX
      XXXXXXXXXXXXXXXXXXXXXXXX
      XXXXXXXXXXXXXXXXXXXXXXXX
      XXXXXXXXXXXXXXXXXXXXXXXX

ncsim: *W,RNQUIE: Simulation is complete.
ncsim> exit
```

3. Pseudocode

Check Command Line

If the number of arguments is incorrect, output error message and terminate

Open ISC File and Verilog File

If unable to open files, output error message and terminate the program

Read ISC File

Read ISC file line by line

Check the boolean variable `is_logicgate_input_statements`

YES:(it's logic gate input)

confirm the logic gate input address is correct (because of `fromType`)

confirm the logic gate input name is correct (gat or gat_out)

Save the data as `gateDescription` in the form of verilog syntax

NO:(it's normal situation)

If it's of type 'from', record the address and fanout, save to `myfromTy`

If it's of type 'inpt', save address to `inputAddress`

If it's a logic gate (`logicGate`) type,

If fanout is 0, save to `outputAddress`

Otherwise, save to `wireAddress`, and mark `is_logicgate_input_stateme`

Generate Verilog Code

Output module name ,Node in `inputAddress` and `outputAddress`

Output Node in `inputAddress`

Output Node in `outputAddress`

Output Node in `wireAddress`

Output description of logic gates stored by `gateDescription`

End

4. My Data Structure and the Program Details

Program Idea

First read the data from isc, then save the data into different data structures, and finally output it into verilog code

```
55 vector<string> inputAddress;
56 vector<string> outputAddress;
57 vector<string> wireAddress;
58 vector<string> gateDescription;
```

Read iscfile

When reading an isc file, read it line by line and save the data as **vector<string> words**

According to the format of isc file,

if this line is describing the input of a logic gate, words[0...n] are all inputs of the logic gate.

If this line is the general case, then

words[0] = address

words[2] = type

words[3] = fansout

Data can be processed in different types.

```
71 | vector<string> words;
72 | string address;
73 | string type;
74 | string fansout;
75 | vector<fromTypeLine> myfromTypeLineRecord;
76 | bool is_logicgate_input_statement = 0;
```

How to deal with the type "from"

Use a set of struct to record the address and fansout when type = from

Ex: 8 8fan from 3gat >sa1 would be stored as {8, 3}

```
22 | struct fromTypeLine {
23 |     string fromTypeLine_address;           //address
24 |     string fromTypeLine_fansout;          //fansout
25 | };
```

Declare a vector of myfromTypeLineRecord to access fromTypeLine encountered in each row

```
59 | vector<fromTypeLine> myfromTypeLineRecord;
```

When type = from is encountered, push the record to myfromTypeLineRecord

Ex:

8 8fan from 3gat >sa1 ———> stored as <{8, 3}>

9 9fan from 3gat >sa1 ———> Stored as <{8, 3}, {9, 3}>

Before outputting into verilog gate descriptor statements, check whether the input of this logic gate is in myfromTypeLineRecord

Ex:

11 11gat nand 2 2 >sa0 >sa1

9 6

-> nand gat11 (gat_out11, gat3, gat6); 9 was replaced by 3 because {9, 3} was in myfromTypeLineRecord

Output Verilog Code

Taking c17.v as example :

```

1  `timescale 1ns/1ps
2  module c17 (gat1, gat2, gat3, gat6, gat7, gat_out22, gat_out23);
3  input gat1, gat2, gat3, gat6, gat7;
4  output gat_out22, gat_out23;
5  wire gat_out10, gat_out11, gat_out16, gat_out19;
6  nand gat10 (gat_out10, gat1, gat3);
7  nand gat11 (gat_out11, gat3, gat6);
8  nand gat16 (gat_out16, gat2, gat_out11);
9  nand gat19 (gat_out19, gat_out11, gat7);
10 nand gat22 (gat_out22, gat_out10, gat_out16);
11 nand gat23 (gat_out23, gat_out16, gat_out19);
12 endmodule

```

The same line-by-line output as reading isc.

The **line 2** uses the stored **inputAddress** and **outputAddress** to output.

The **line 3** uses the stored **inputAddress** to output.

The **line 4** uses the stored **outputAddress** to output.

The **line 5** uses the stored **wireAddress** to output.

The **line 6 to line 11** uses the stored **gateDescription** to output.

5. The Hardness I Encountered

How to store the data of From type?

My final method is to add a new struct and use vector to store all the From type I encountered, Then use a forloop to do the query, the time complexity is $O(n)$.

There are two data structures below that I wanted to use before:

Example:

8 8fan from 3gat >sa1

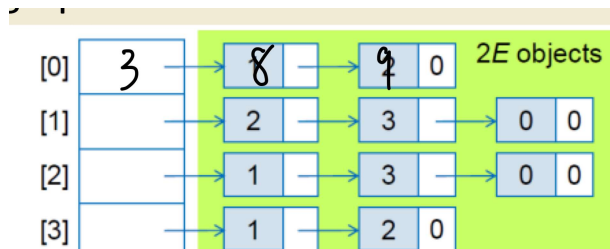
9 9fan from 3gat >sa1

1. Adjacency list

Save it in the following way. As long as it encounters 8 or 9, it will return the first value in the sequence, which is 3

This way you can compare with my results and save less data.

However, after reading the source code on the Internet, most people declare a fixed-size array at the beginning of this 2-dimensional data structure in the vertical direction, and it is difficult to use dynamic configuration (vector) to do, so I didn't use it in the end.



2. Hash table

Although the time complexity of searching for data is only **O(1)**, it cannot deal with duplicate data, which is also called **hash collision**.

After {3,8} is saved, any further {3,9} {3,8} will be overwritten.

6. My Suggestion

I think the difficulty level of the assignment is okay, not too difficult, so I have no suggestions for the assignment.

But I hope that if possible, the flipped classroom can be uploaded to eeclase a few days earlier, giving us more time to make arrangements. By the way, the sounds at the end of the flipped classroom are a bit too loud, and I get scared every time.

7. Reference

Adjency List (<https://ithelp.ithome.com.tw/articles/10246151>)

Hash Table ([https://medium.com/@ralph-](https://medium.com/@ralph-tech/%E8%B3%87%E6%96%99%E7%B5%90%E6%A7%8B%E5%AD%B8%E7%BF%92%E7%AD%86%E8%A8%98-%E9%9B%9C%E6%B9%8A%E8%A1%A8-hash-table-15f490f8ede6)

[tech/%E8%B3%87%E6%96%99%E7%B5%90%E6%A7%8B%E5%AD%B8%E7%BF%92%E7%AD%86%E8%A8%98-%E9%9B%9C%E6%B9%8A%E8%A1%A8-hash-table-15f490f8ede6](https://medium.com/@ralph-tech/%E8%B3%87%E6%96%99%E7%B5%90%E6%A7%8B%E5%AD%B8%E7%BF%92%E7%AD%86%E8%A8%98-%E9%9B%9C%E6%B9%8A%E8%A1%A8-hash-table-15f490f8ede6))